MICROSOFT DYNAMICS 365 BUSINESS CENTRAL

# HỌC LẬP TRÌNH AL

## Từ nền tảng đến xây dựng extension ERP

Giáo trình tự học bằng tiếng Việt • Theo 10 học phần trong Microsoft Learn

Giải thích ngắn gọn nhưng có chiều sâu • Ví dụ AL thực hành • Bài tập & đáp án

**ĐỐI TƯỢNG**
Người mới học AL, sinh viên CNTT, thực tập sinh ERP hoặc lập trình viên chuyển sang Business Central. Chỉ cần hiểu cơ bản về biến, hàm, điều kiện và dữ liệu.

**Biên soạn:** Tài liệu học tập độc lập dựa trên nội dung và cấu trúc của Microsoft Learn. Không phải tài liệu do Microsoft xuất bản.

**Ngày biên soạn:** 02/11/2026 • Ngôn ngữ: Tiếng Việt • Ví dụ AL có tính minh họa và cần chạy thử trên sandbox.

# HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

Đây là tài liệu giải thích lại 10 mô-đun trong lộ trình **Learn the application foundation with the AL programming language for Microsoft Dynamics 365 Business Central**, không phải bản dịch nguyên văn. Mục tiêu là hiểu AL hoạt động như thế nào trong một hệ thống ERP, biết đọc mã và viết extension nhỏ có thể triển khai trên môi trường thử nghiệm.

- Đọc lần lượt chương 0–10; chương 11 là dự án thực hành cuối khóa.
- Sau từng ví dụ, xác định: đối tượng AL nào? Chạy khi nào? Đọc/ghi bảng nào? Có rủi ro dữ liệu không?
- Các đoạn mã dùng ID ví dụ 50100–50199; đổi theo dải ID được cấp và kiểm tra trùng trước khi publish.
- Luôn thử CRUD, event subscriber, xóa/sửa và dịch thuật ở **sandbox**, không thử trực tiếp trên dữ liệu thật.
- Các tính năng và cấu hình của Business Central thay đổi theo phiên bản; nếu khác với màn hình thực tế, ưu tiên tài liệu Microsoft Learn hiện hành.

## Bảng lộ trình và mục tiêu đầu ra

| Phần | Chủ đề | Kỹ năng sau khi học |
|---|---|---|
| 0 | Môi trường & tư duy ERP | Tạo AL project, biết các loại object |
| 1 | Ngôn ngữ AL căn bản | Biến, kiểu, enum, collection, biểu thức |
| 2 | Câu lệnh điều khiển | If, case, vòng lặp, comment |
| 3 | Hàm tích hợp | Chuỗi, ngày, số, list, system |
| 4 | Xử lý lỗi | Error, TryFunction, ErrorInfo, UI |
| 5 | Procedure | Phạm vi, tham số, return, var |
| 6 | Codeunit | Tách và tái sử dụng nghiệp vụ |
| 7 | Triggers & events | Mở rộng theo sự kiện |
| 8 | Interfaces | Thay thế chiến lược xử lý |
| 9 | Dữ liệu | Đọc, lọc, ghi, validate, FlowField |
| 10 | Đa ngôn ngữ | XLF, caption, translation |
| 11 | Dự án cuối khóa | Extension kiểm tra dữ liệu giao nhận |

# Mục lục

**HƯỚNG DẪN SỬ DỤNG TÀI LIỆU 2**

Bảng lộ trình và mục tiêu đầu ra . . . 2
Mục lục . . . 3

**0. BẮT ĐẦU: AL TRONG HỆ THỐNG ERP 6**

0.1. AL là gì? . . . 6
0.2. Chuẩn bị môi trường làm việc . . . 6
0.3. Bộ khung một extension AL . . . 6
0.4. Quy trình phát triển chuẩn . . . 7

**1. CĂN BẢN AL: KIỂU DỮ LIỆU VÀ BIỂU THỨC 8**

1.1. Cú pháp và định danh . . . 8
1.2. Scope: biến local và global . . . 8
1.3. Những kiểu dữ liệu quan trọng . . . 9
1.4. Option và enum: nên ưu tiên enum . . . 9
1.5. Array, List và Dictionary . . . 9
1.6. Gán, chuyển kiểu và biểu thức . . . 10

**2. CÂU LỆNH: ĐIỀU KIỆN VÀ VÒNG LẶP 11**

2.1. Comments và compound statements . . . 11
2.2. If / else . . . 11
2.3. Case: nhiều nhánh theo cùng một giá trị . . . 11
2.4. Các kiểu vòng lặp . . . 11
2.5. Duyệt Record bằng FindSet + repeat . . . 12
2.6. Nhìn điều khiển luồng theo nghiệp vụ . . . 12

**3. HÀM TÍCH HỢP CỦA AL 13**

3.1. Tương tác với người dùng . . . 13
3.2. Các hàm chuỗi . . . 13
3.3. Hàm ngày giờ . . . 13
3.4. Hàm số và làm tròn . . . 13
3.5. Mảng, List, Dictionary . . . 14
3.6. Hàm hệ thống và hàm trên biến . . . 14

**4. XỬ LÝ LỖI VÀ TRẢI NGHIỆM NGƯỜI DÙNG 15**

4.1. Lỗi là một phần của quy tắc nghiệp vụ . . . 15
4.2. TryFunction: bắt lỗi, không phải rollback . . . 15

4.3. Thu gom nhiều lỗi với ErrorInfo . . . 15
4.4. Dialog, Message, Confirm và GUI . . . 16
4.5. Checklist trước khi phát hành . . . 16

**5. PROCEDURE: TÁI SỬ DỤNG VÀ PHÂN RÃ NGHIỆP VỤ 17**

5.1. Một procedure nên làm đúng một việc . . . 17
5.2. Các phạm vi truy cập . . . 17
5.3. Truyền tham số by value và by reference . . . 17
5.4. Return và Exit . . . 17
5.5. Tách logic tốt hơn: ví dụ . . . 18

**6. CODEUNIT: LỚP LOGIC CỦA EXTENSION 19**

6.1. Codeunit là gì? . . . 19
6.2. Gọi procedure từ codeunit khác . . . 19
6.3. OnRun khác với procedure tự định nghĩa . . . 19
6.4. Codeunit và phân quyền . . . 19
6.5. Những nguyên tắc tổ chức tốt . . . 19

**7. TRIGGERS, EVENTS VÀ KIẾN TRÚC MỞ RỘNG 21**

7.1. Trigger: tự chạy ở thời điểm nhất định . . . 21
7.2. OnValidate và Validate . . . 21
7.3. Tableextension và pageextension . . . 21
7.4. Kiến trúc event: publisher và subscriber . . . 22
7.5. Tự định nghĩa một event . . . 22
7.6. Đăng ký subscriber . . . 22
7.7. Event cho ứng dụng chuẩn . . . 22

**8. INTERFACE: THAY THẾ CÁCH TRIỂN KHAI 24**

8.1. Interface giải quyết vấn đề gì? . . . 24
8.2. Khai báo interface và triển khai . . . 24
8.3. Dùng cùng một kiểu để gọi các thuật toán khác nhau . . . 24
8.4. Interface và enum có thể kết hợp . . . 25
8.5. Khi nào chưa cần interface? . . . 25

**9. THAO TÁC DỮ LIỆU: RECORD VÀ CƠ SỞ DỮ LIỆU 26**

9.1. Dùng Record để làm việc với bảng . . . 26
9.2. Get, FindFirst, FindLast, FindSet và IsEmpty . . . 26
9.3. SetRange, SetFilter, SetCurrentKey . . . 26
9.4. Thêm dữ liệu với Init, Validate, Insert . . . 26

9.5. Modify, ModifyAll, Delete, DeleteAll . . . 27
9.6. TestField, FieldError, Validate . . . 27
9.7. FlowField, CalcFields, CalcSums . . . 27
9.8. Khóa dữ liệu và hiệu năng . . . 27

**10. PHÁT TRIỂN ĐA NGÔN NGỮ 29**

10.1. Vì sao extension cần translation? . . . 29
10.2. Caption và Label . . . 29
10.3. Cấu hình TranslationFile và sinh tệp XLF . . . 29
10.4. Cấu trúc tối giản của bản dịch XLF . . . 29
10.5. Dịch tự động và kiểm tra chất lượng . . . 29

**11. DỰ ÁN CUỐI KHÓA: DELIVERY REVIEW 31**

11.1. Bài toán nghiệp vụ . . . 31
11.2. Cấu trúc thư mục đề xuất . . . 31
11.3. File 1 - Enum trạng thái . . . 31
11.4. File 2 - Table lưu yêu cầu . . . 32
11.5. File 3 - Codeunit kiểm tra và duyệt . . . 32
11.6. File 4 - Page nhập và duyệt . . . 33
11.7. File 5 - Subscriber minh họa . . . 35
11.8. Quy trình chạy thử từng bước . . . 35
11.9. Tiêu chí tự chấm điểm . . . 35

**12. TỰ KIỂM TRA, LỖI THƯỜNG GẶP & LỘ TRÌNH HỌC 36**

12.1. Mười lỗi người mới thường gặp . . . 36
12.2. Câu hỏi ôn tập tổng hợp . . . 36
12.3. Đáp án ngắn . . . 37
12.4. Kế hoạch 14 ngày để học chắc . . . 38
12.5. Thuật ngữ tiếng Anh - tiếng Việt . . . 38

**NGUỒN HỌC VÀ CÁCH ĐỌC TIẾP 40**

Sau tài liệu này nên học gì? . . . 40

# 0. BẮT ĐẦU: AL TRONG HỆ THỐNG ERP

## 0.1. AL là gì?

AL (Application Language) là ngôn ngữ phát triển extension cho Dynamics 365 Business Central, một hệ thống ERP hỗ trợ các quy trình như tài chính - kế toán, mua hàng, bán hàng, kho và vận hành. AL không phải SQL và cũng không phải JavaScript: bạn lập trình trực tiếp vào các đối tượng nghiệp vụ trên nền tảng Business Central.

Một extension là gói ứng dụng chứa các đối tượng mới và những phần mở rộng đối tượng sẵn có. Bạn có thể thêm trường trên khách hàng, tạo biểu mẫu nhập dữ liệu, thêm quy tắc kiểm tra hoặc phản ứng trước một sự kiện mà không cần chỉnh sửa trực tiếp mã nền tảng của Microsoft.

| Đối tượng AL | Hiểu đơn giản | Ví dụ trong ERP |
|---|---|---|
| table / tableextension | Cấu trúc lưu dữ liệu / bổ sung trường | Theo dõi mã vận đơn |
| page / pageextension | Giao diện nhập, xem dữ liệu | Màn hình danh sách yêu cầu |
| codeunit | Đóng gói logic, thủ tục | Kiểm tra trước khi duyệt |
| enum / enumextension | Danh sách giá trị trạng thái | Mới, Đang xử lý, Hoàn thành |
| interface | Hợp đồng phương thức | Nhiều thuật toán tính phí |
| event subscriber | Bắt sự kiện để chạy mã | Cảnh báo khi đổi trạng thái |

## 0.2. Chuẩn bị môi trường làm việc

- Cài Visual Studio Code và extension **AL Language** từ Microsoft.
- Có tài khoản/đường truy cập Business Central sandbox; ưu tiên sandbox đám mây hoặc môi trường container được đội kỹ thuật cấp.
- Trong VS Code mở Command Palette: **AL: Go!** để tạo dự án; chọn máy chủ/loại sandbox phù hợp.
- Mở app.json (metadata extension), launch.json (cấu hình kết nối và chạy), thư mục .alpackages (symbol package).
- Chạy **AL: Download Symbols** để tải định nghĩa đối tượng; sau đó tạo file .al, dùng **Ctrl+F5** để publish không debug hoặc **F5** để publish và debug (tùy cấu hình).

> **Lưu ý quan trọng**
> Nếu lệnh tải symbols hoặc publish thất bại, hãy kiểm tra **authentication, tenant, environment, version, app.json, launch.json và quyền truy cập sandbox**. Đừng "sửa đại" thông số production. Quyền và quy trình triển khai phụ thuộc tổ chức.

## 0.3. Bộ khung một extension AL

```
codeunit 50100 "AL Hello Demo"
{
    trigger OnRun()
    begin
        Message('Xin chao Business Central!');
    end;
}
```

Giải thích: **codeunit 50100** là một đối tượng mã; **OnRun** là điểm vào khi codeunit được chạy; **Message** hiển thị thông báo trong giao diện có người dùng. Bài này chưa đụng tới dữ liệu và phù hợp để kiểm tra việc publish.

## 0.4. Quy trình phát triển chuẩn

**Nghiệp vụ  mô hình dữ liệu  giao diện  validation  logic codeunit  test  xuất bản extension**. Không nên mở VS Code và viết trigger ngay khi chưa hiểu các trạng thái, khóa chính, quyền và trường hợp ngoại lệ của quy trình.

### Tự kiểm tra

- Table khác page ở điểm nào?
- Vì sao nên đóng gói tính toán vào codeunit thay vì sao chép trên nhiều page?

**Gợi ý đáp án:** Table lưu thông tin và ràng buộc dữ liệu; page trình bày và tương tác. Codeunit giúp một quy tắc được tái sử dụng, kiểm thử và bảo trì nhất quán.

# 1. CĂN BẢN AL: KIỂU DỮ LIỆU VÀ BIỂU THỨC

## 1.1. Cú pháp và định danh

AL dùng dấu **;** kết thúc câu lệnh/khai báo, **{ }** để bao phần thân đối tượng, **begin … end** để bao các câu lệnh, **:=** để gán và **//** cho chú thích một dòng. Ngôn ngữ không phân biệt chữ hoa/chữ thường trong tên, nhưng nên dùng PascalCase để dễ đọc.

Tên đối tượng/biến chứa dấu cách cần đặt trong dấu nháy kép, ví dụ **"Shipment Status"**. Văn bản literal đặt trong dấu nháy đơn, ví dụ **'Pending'**. ID đối tượng phải thuộc dải được phép và không trùng.

```
procedure CalculateTotal(Quantity: Decimal; Price: Decimal): Decimal
var
    Total: Decimal;
begin
    // Gan gia tri cho bien
    Total := Quantity * Price;
    exit(Total);
end;
```

## 1.2. Scope: biến local và global

Biến khai báo trong thủ tục chỉ được sử dụng trong thủ tục đó (local). Biến khai báo ở cấp đối tượng có thể được các procedure và trigger trong chính đối tượng tham chiếu (global). Phạm vi rộng tiện lợi nhưng dễ làm phát sinh trạng thái khó theo dõi, vì vậy mặc định nên dùng biến local.

```
codeunit 50101 "Counter Demo"
{
    var
        TotalCalls: Integer; // Bien cap object

    procedure Increase(): Integer
    var
        Step: Integer; // Bien local
    begin
        Step := 1;
        TotalCalls += Step;
        exit(TotalCalls);
    end;
}
```

Các biến hệ thống hay gặp: **Rec** (bản ghi hiện tại), **xRec** (giá trị bản ghi trước thay đổi trong một số ngữ cảnh), **CurrPage** và **CurrReport**. Không nhầm xRec với bản ghi lịch sử luôn được lưu trong cơ sở dữ liệu.

## 1.3. Những kiểu dữ liệu quan trọng

| Kiểu | Ý nghĩa | Ví dụ và lưu ý |
|---|---|---|
| Integer / BigInteger | Số nguyên | Số lượng hộp, số thứ tự |
| Decimal | Số thập phân | Giá, chiết khấu, cân nặng; ưu tiên Decimal cho tiền |
| Boolean | Đúng / sai | IsApproved := true |
| Text[n] | Chuỗi ký tự | Tên hàng, ghi chú; có phân biệt hoa/thường |
| Code[n] | Mã nghiệp vụ | Mã đối tác; chuẩn hóa chữ hoa và khoảng trắng |
| Date / Time / DateTime | Ngày / giờ | ShipmentDate, ModifiedAt |
| Duration | Khoảng thời gian | Thời gian xử lý |
| Record Customer | Bản ghi của một bảng | Đọc/ghi dữ liệu khách hàng |
| List / Dictionary / array | Tập hợp nhiều phần tử | Danh sách mã kho, ánh xạ mãtên |
| Enum | Giá trị phân loại có tên | Trạng thái vận đơn |

Giá trị khởi tạo mặc định: số bằng 0, Boolean là false, chuỗi rỗng, ngày chưa xác định là 0D. Việc “mặc định” không đồng nghĩa đã có dữ liệu hợp lệ; phải kiểm tra trường bắt buộc.

## 1.4. Option và enum: nên ưu tiên enum

Option là danh sách giá trị định nghĩa ngay tại nơi sử dụng và có hạn chế về tái sử dụng/mở rộng. Enum là đối tượng độc lập chứa các giá trị có ID và caption, có thể tái sử dụng trong biến, trường và tham số. Enum có **Extensible = true** thì một extension khác mới bổ sung được giá trị.

```
enum 50110 "Delivery Stage"
{
    Extensible = true;

    value(0; Draft) { Caption = 'Draft'; }
    value(1; InTransit) { Caption = 'In transit'; }
    value(2; Delivered) { Caption = 'Delivered'; }
}
```

Trong mã: **CurrentStage: Enum "Delivery Stage";**; kiểm tra bằng **CurrentStage = CurrentStage::Delivered**. Không coi thứ tự ID enum là thứ tự ưu tiên nghiệp vụ; hãy viết quy tắc rõ ràng.

## 1.5. Array, List và Dictionary

| Cấu trúc | Khi nào dùng | Điểm cần nhớ |
|---|---|---|
| array[3] of Decimal | Số phần tử cố định | Chỉ số đầu tiên là 1 |
| List of [Text] | Danh sách dài thay đổi | Add, Get, Count, Contains |
| Dictionary of [Code[20], Decimal] | Ánh xạ khóa–giá trị | Key duy nhất; cần kiểm tra tồn tại |

```
procedure CollectionExample()
var
    Amounts: array[3] of Decimal;
    Locations: List of [Code[10]];
    RateByZone: Dictionary of [Code[10], Decimal];
    Rate: Decimal;
begin
    Amounts[1] := 125000;
    Locations.Add('HN');
    Locations.Add('HCM');
    RateByZone.Add('HN', 15000);
    if RateByZone.Get('HN', Rate) then
        Message('Phi khu vuc: %1', Rate);
end;
```

## 1.6. Gán, chuyển kiểu và biểu thức

AL sử dụng **:=** để gán, **=** để so sánh. Các phép toán số học thông dụng: +, -, *, /, div, mod; so sánh: =, <>, <, >, <=, >=; logic: and, or, not. Luôn dùng dấu ngoặc để ưu tiên biểu thức phức tạp.

```
TotalAmount := UnitPrice * Quantity;
HasPriority := (Quantity &gt;= 10) and (UnitPrice &gt; 0);
Counter += 1;
if not HasPriority then
    Message('Don hang thong thuong');
```

Chuyển đổi thông dụng: **Format** biến giá trị thành văn bản; **Evaluate** phân tích chuỗi thành kiểu đích; hàm trả Boolean giúp tránh lỗi khi dữ liệu đầu vào không hợp lệ. Kiểu Code tự chuẩn hóa chữ hoa và bỏ khoảng trắng đầu/cuối, nên cần suy nghĩ khi mã được sinh từ hệ thống khác.

```
procedure ParseQuantity(InputText: Text): Integer
var
    Quantity: Integer;
begin
    if not Evaluate(Quantity, InputText) then
        Error('So luong khong hop le: %1', InputText);
    exit(Quantity);
end;
```

> **Lỗi thường gặp**
> Không dùng giá trị text như thể luôn chuyển được sang số. Không dùng Decimal để lưu mã khách hàng dù mã toàn chữ số. Không nhầm **1-based** của array/list với 0-based của nhiều ngôn ngữ khác.

### Tự kiểm tra

- Vì sao Code[20] hợp lý cho “mã kho” hơn Decimal?
- Nếu List có hai phần tử, chỉ số lấy phần tử đầu tiên là bao nhiêu?
- Điều gì xảy ra khi Evaluate(IntegerVar, 'abc') không thành công?

**Gợi ý đáp án:** Mã là định danh, không phải số để tính toán; phần tử đầu có chỉ số 1; Evaluate trả false nếu gọi theo điều kiện Boolean.

# 2. CÂU LỆNH: ĐIỀU KIỆN VÀ VÒNG LẶP

## 2.1. Comments và compound statements

Chú thích một dòng dùng //, chú thích nhiều dòng dùng /* ... */. Một compound statement là nhiều câu lệnh đặt trong **begin ... end** và được xem như một khối. Nếu sau if/else/for cần chạy nhiều câu lệnh, bạn phải nhóm khối bằng begin ... end.

```
if Quantity &gt; 0 then begin
    Amount := Quantity * UnitPrice;
    Message('Thanh tien: %1', Amount);
end else
    Error('So luong phai lon hon 0');
```

## 2.2. If / else

If kiểm tra biểu thức Boolean. Chỉ đặt dấu ; đúng vị trí: không thêm dấu ; ngay trước **else** thuộc cùng một if. Khi có nhiều điều kiện phức tạp, tách thành procedure nhỏ giúp dễ kiểm thử.

```
if Weight &lt;= 0 then
    Error('Khoi luong khong hop le')
else if Weight &lt;= 2 then
    Fee := 15000
else
    Fee := 25000;
```

## 2.3. Case: nhiều nhánh theo cùng một giá trị

Case thích hợp khi quyết định theo trạng thái enum, loại chứng từ hoặc mã phương thức. Cần nhánh else để xử lý giá trị không mong đợi, đặc biệt với enum có thể được mở rộng.

```
case Stage of
    Stage::Draft:
        Message('Cho duyet');
    Stage::InTransit:
        Message('Dang van chuyen');
    Stage::Delivered:
        Message('Da giao');
    else
        Error('Trang thai chua duoc ho tro');
end;
```

## 2.4. Các kiểu vòng lặp

| Câu lệnh | Ứng dụng | Điểm cần lưu ý |
|---|---|---|
| for ... to / downto | Lặp với bộ đếm biết trước | Không tự ý sửa bộ đếm bên trong |
| while ... do | Lặp khi điều kiện còn đúng | Cẩn thận vòng lặp vô hạn |
| repeat ... until | Chạy ít nhất một lần | Kết thúc khi điều kiện thành true |
| foreach ... in | Duyệt phần tử của collection | Thuận tiện cho List, Dictionary key |

```
for Counter := 1 to 3 do
    Message('Lan %1', Counter);

while Quantity &gt; 0 do
    Quantity -= 1;

repeat
    Attempts += 1;
until (Attempts &gt;= 3) or IsValid;
```

## 2.5. Duyệt Record bằng FindSet + repeat

Khác với vòng lặp mảng, bản ghi của Business Central thường được lọc trước rồi duyệt bằng FindSet và Next. Mọi **FindSet** nên có điều kiện kiểm tra Boolean, vì bộ dữ liệu có thể rỗng.

```
Customer.SetRange(Blocked, Customer.Blocked::" ");
if Customer.FindSet() then
    repeat
        // Xu ly tung khach hang
        CustomerCount += 1;
    until Customer.Next() = 0;
```

Ví dụ dùng trường Blocked với giá trị rỗng chỉ mang tính minh họa. Khi đưa vào dự án thật, cần kiểm tra kiểu enum/option của trường theo symbols đang sử dụng.

## 2.6. Nhìn điều khiển luồng theo nghiệp vụ

Một quy trình duyệt yêu cầu giao hàng có thể mô tả: (1) kiểm tra mã khách và ngày; (2) nếu lỗi thì Error; (3) nếu hợp lệ thì chọn phí theo khu vực; (4) lưu trạng thái; (5) phát sự kiện. Hãy ưu tiên biểu diễn nghiệp vụ rõ ràng thay vì chồng nhiều if trong một trigger.

### Tự kiểm tra

- Khi nào dùng case thay vì nhiều if?
- Đoạn repeat ... until thực thi ít nhất mấy lần?
- Vì sao phải kiểm tra if Record.FindSet() trước khi repeat?

**Gợi ý đáp án:** Case rõ hơn khi nhiều nhánh dựa trên cùng giá trị; repeat thực thi tối thiểu một lần; FindSet có thể không tìm được bản ghi, khi đó không được duyệt như thể đã có dữ liệu.

# 3. HÀM TÍCH HỢP CỦA AL

## 3.1. Tương tác với người dùng

| Hàm | Vai trò | Nên dùng khi |
|---|---|---|
| Message | Thông báo thông tin | Tác vụ tương tác đơn giản |
| Confirm | Yêu cầu Yes/No | Xác nhận trước thao tác nhạy cảm |
| Error | Ngừng xử lý vì lỗi | Sai dữ liệu, vi phạm quy tắc |
| StrMenu | Chọn một phương án | Tác vụ đơn giản, có GUI |
| Dialog.Open / Update / Close | Hiển thị tiến độ | Quy trình có giao diện dài |

```
if not Confirm('Ban muon tiep tuc?', false) then
    exit;
Message('Da xac nhan');
```

Trong job queue, API và background session có thể không có giao diện đồ họa. Chỉ gọi các phương thức yêu cầu người dùng khi **GuiAllowed** là true, và không dùng Message làm cơ chế ghi log vận hành.

## 3.2. Các hàm chuỗi

Thường gặp: **StrLen** đo chiều dài; **CopyStr** lấy chuỗi con; **DelStr** xóa đoạn; **InsStr** chèn; **UpperCase/LowerCase** đổi kiểu chữ; **StrPos** tìm vị trí; **StrSubstNo** thay placeholder trong chuỗi. Chọn giới hạn độ dài rõ ràng cho trường Text[n].

```
FullCode := 'hn-warehouse-01';
ShortCode := CopyStr(UpperCase(FullCode), 1, 10);
CaptionText := StrSubstNo('Ma kho: %1', ShortCode);
if StrPos(FullCode, 'warehouse') &gt; 0 then
    Message(CaptionText);
```

Các chữ có dấu tiếng Việt được phép ở kiểu Text, nhưng khi so sánh và chuyển đổi chữ hoa/thường nên kiểm tra ngữ cảnh ngôn ngữ/đối chiếu. Mã tích hợp API tốt nhất vẫn nên theo chuẩn đã quy định.

## 3.3. Hàm ngày giờ

**Today** cho ngày hiện tại; **CurrentDateTime** trả DateTime; **CalcDate** tính toán theo công thức ngày; **WorkDate** là ngày làm việc được người dùng thiết lập và có thể khác hôm nay; **DMY2Date** tạo một ngày từ ngày/tháng/năm. Khi xử lý kế toán, phân biệt Today và WorkDate rất quan trọng.

```
DueDate := CalcDate('&lt;+7D&gt;', Today());
MonthEnd := CalcDate('&lt;CM&gt;', WorkDate());
CutoffDate := DMY2Date(1, 1, 2027);
```

Nếu công thức ngày hoặc logic kỳ kế toán phụ thuộc locale, sử dụng dạng date formula tiêu chuẩn với dấu góc như ví dụ và kiểm thử trên tenant mục tiêu.

## 3.4. Hàm số và làm tròn

Các hàm cơ bản gồm **Abs**, **Round**, **Power** và một số hàm lượng giác. Với tiền tệ, dùng chính sách làm tròn của nghiệp vụ và đơn vị tiền thay vì tự mặc định luôn làm tròn 2 chữ số.

```
NetAmount := UnitPrice * Quantity;
RoundedAmount := Round(NetAmount, 0.01, '=');
Difference := Abs(NetAmount - RoundedAmount);
```

## 3.5. Mảng, List, Dictionary

Array có thể dùng **ArrayLen** và **CopyArray**. Với List, các method như **Add, Contains, Count, Get, Remove** giúp thao tác linh hoạt. Với Dictionary, **Add, ContainsKey, Get, Set** phù hợp tra cứu theo khóa. Kiểm tra tồn tại khóa trước khi đọc dữ liệu không đảm bảo có giá trị.

```
Zones.Add('NORTH');
Zones.Add('SOUTH');
if Zones.Contains('NORTH') then
    Message('Co khu vuc NORTH');

if not Fees.ContainsKey('NORTH') then
    Fees.Add('NORTH', 18000);
```

## 3.6. Hàm hệ thống và hàm trên biến

Một số nhóm hay dùng: **CompanyName, UserId, GuiAllowed, CreateGuid, Format, Evaluate, Clear**. Trong đối tượng bản ghi sẽ có các method riêng như **Init, Get, Validate, Insert**; đừng nhầm hàm global với method của Record.

```
Clear(Amount);
ShipmentGuid := CreateGuid();
if GuiAllowed then
    Message('Cong ty: %1', CompanyName());
```

> **Quy tắc hiệu năng**
> Không đặt Message trong vòng lặp hàng nghìn bản ghi, không xử lý chuỗi phức tạp trên mỗi bản ghi nếu có thể tối ưu lọc trên máy chủ và không dùng từng FindFirst trong vòng lặp khi có cách truy vấn tập hợp tốt hơn.

### Tự kiểm tra

- Đối với công việc kế toán, Today và WorkDate có thể khác nhau không?
- Khi nhận “125.50” từ CSV, nên dùng thủ tục chuyển kiểu nào?
- Có nên dùng Confirm trong web service?

**Gợi ý đáp án:** Có; dùng Evaluate và kiểm tra kết quả/locale; không - web service không có UI nên phải thiết kế đầu vào/cách phản hồi riêng.

# 4. XỬ LÝ LỖI VÀ TRẢI NGHIỆM NGƯỜI DÙNG

## 4.1. Lỗi là một phần của quy tắc nghiệp vụ

Trong ERP, dữ liệu sai có thể gây hậu quả lớn hơn lỗi giao diện: ghi sổ nhầm, tồn kho sai, giao hàng thiếu hoặc mất đối soát. Vì vậy, lỗi cần nêu rõ trường nào sai, lý do và hành động khắc phục. Các cách quan trọng là **Error**, **TestField**, **FieldError**, **TryFunction** và **ErrorInfo**.

```
if Quantity &lt;= 0 then
    Error('So luong phai lon hon 0');

Shipment.TestField("Customer No.");
if Shipment."Weight (Kg)" &lt; 0 then
    Shipment.FieldError("Weight (Kg)",
        'khong the am');
```

## 4.2. TryFunction: bắt lỗi, không phải rollback

Gắn attribute **[TryFunction]** cho procedure khi cần thử một phép có thể thất bại. Nếu gọi hàm qua **if MyTryFunction() then...**, lỗi có thể được bắt và procedure trả true/false. Nếu bỏ qua kết quả trả về, lỗi có thể truyền như bình thường. **Đặc biệt: các thao tác ghi dữ liệu được thực hiện bên trong TryFunction không tự động rollback**; nên dùng TryFunction cho các bước đọc/xử lý thuần, tránh các giao dịch ghi phức tạp.

```
procedure SafeReadNumber(InputText: Text): Boolean
begin
    if TryReadNumber(InputText) then
        exit(true);

    if GuiAllowed then
        Message('Khong doc duoc gia tri: %1', InputText);
    exit(false);
end;
```

```
[TryFunction]
local procedure TryReadNumber(InputText: Text)
var
    Value: Integer;
begin
    if not Evaluate(Value, InputText) then
        Error('Invalid integer');
end;
```

Trong ví dụ đọc số, **Evaluate** tự trả Boolean nên không bắt buộc TryFunction. Mục đích minh họa cơ chế TryFunction; khi làm thật chọn giải pháp đơn giản nhất.

## 4.3. Thu gom nhiều lỗi với ErrorInfo

Khi kiểm tra một chứng từ có nhiều trường, dừng ngay ở lỗi đầu khiến người dùng phải sửa từng lần. Cơ chế **collectible errors** cho phép thu gom nhiều lỗi, thường kết hợp **ErrorInfo.Create(..., true)** và attribute **[ErrorBehavior(ErrorBehavior::Collect)]**. Không dùng cơ chế này như giấy phép để tiếp tục ghi dữ liệu nửa chừng; cần thiết kế transaction và rollback rõ ràng.

```
[ErrorBehavior(ErrorBehavior::Collect)]
procedure ValidateInput(CustomerNo: Code[20]; Weight: Decimal)
begin
    if CustomerNo = '' then
        Error(ErrorInfo.Create('Thieu ma khach', true));
    if Weight &lt;= 0 then
        Error(ErrorInfo.Create('Can nang phai &gt; 0', true));
end;
```

Một procedure hoặc phạm vi gọi có thể hiển thị danh sách lỗi đã thu gom. Những phần liên quan đến **HasCollectedErrors, GetCollectedErrors, ClearCollectedErrors** nên thực hành trong sandbox và theo tài liệu phiên bản đang chạy.

## 4.4. Dialog, Message, Confirm và GUI

Message là phản hồi thông tin, Confirm yêu cầu quyết định, Error chặn thao tác, Dialog giúp hiển thị tiến độ. Chỉ dùng giao diện khi có người tương tác. Với API/job queue, trả cấu trúc kết quả/lỗi phù hợp và ghi telemetry thay vì bật hộp thoại.

## 4.5. Checklist trước khi phát hành

- Lỗi phải có tên trường/mã chứng từ hoặc điều kiện cần sửa; tránh thông báo chung chung “Failed”.
- Giữ message hướng tới người dùng, không đưa thông tin bí mật vào log hoặc lỗi.
- Các thao tác ghi hàng loạt phải có phương án kiểm thử, rollback và phục hồi phù hợp.
- Không nuốt lỗi bằng try rồi giả vờ quy trình thành công.
- Không dùng Message/Confirm ở luồng API hoặc chạy nền nếu không kiểm tra GuiAllowed.

### Tự kiểm tra

- TryFunction có tự động hoàn tác dữ liệu đã ghi không?
- Với 20 dòng dữ liệu nhập có 5 lỗi, nên cân nhắc cơ chế nào?
- Dùng FieldError khác Error ở điểm nào?

**Gợi ý đáp án:** Không; cân nhắc ErrorInfo collectable để báo nhiều lỗi; FieldError gắn lỗi với một trường để người dùng dễ xác định vị trí.

# 5. PROCEDURE: TÁI SỬ DỤNG VÀ PHÂN RÃ NGHIỆP VỤ

## 5.1. Một procedure nên làm đúng một việc

Procedure tương tự một hàm trong nhiều ngôn ngữ lập trình. Nó nhận tham số, chạy logic và có thể trả giá trị. Tách các việc **kiểm tra  tính toán  lưu** thành nhiều procedure giúp mã dễ đọc, dễ tái sử dụng và dễ kiểm thử, đặc biệt khi cùng quy tắc được gọi từ page, API, job và posting.

```
procedure CalculateFreight(Weight: Decimal;
                           UnitRate: Decimal): Decimal
begin
    if Weight &lt;= 0 then
        Error('Can nang khong hop le');
    if UnitRate &lt; 0 then
        Error('Don gia khong hop le');
    exit(Round(Weight * UnitRate, 0.01));
end;
```

## 5.2. Các phạm vi truy cập

| Khai báo | Ai có thể sử dụng? | Lời khuyên |
|---|---|---|
| procedure Foo() | Public theo ngữ cảnh object | Chỉ công khai API thật cần |
| local procedure Foo() | Trong object khai báo | Nên là mặc định cho helper |
| internal procedure Foo() | Trong cùng extension | Cho các phần nội bộ tái sử dụng |
| protected procedure Foo() | Object chủ và extension của nó | Dành cho điểm mở rộng có chủ ý |

Khái niệm “global procedure” khác “global variable”: một procedure public được object khác gọi qua codeunit/page theo quy tắc quyền truy cập; một biến global của object không tự động cho object bên ngoài đọc.

## 5.3. Truyền tham số by value và by reference

Tham số thông thường của kiểu đơn giản được truyền theo giá trị: việc đổi bên trong hàm không đổi biến phía gọi. Thêm từ khóa **var** để truyền theo tham chiếu: procedure được phép sửa chính biến phía gọi. Đây là cách hữu ích cho output nhưng cũng dễ gây side effect.

```
procedure ApplyDiscount(var Amount: Decimal;
                        DiscountPercent: Decimal)
begin
    if (DiscountPercent &lt; 0) or
       (DiscountPercent &gt; 100) then
        Error('Ty le giam gia khong hop le');
    Amount := Amount * (100 - DiscountPercent) / 100;
end;
```

Gọi: **ApplyDiscount(Total, 10);** sẽ thay đổi Total. Nếu thay var bằng truyền giá trị, Total phía gọi không bị thay đổi. Chỉ dùng var khi người gọi thực sự mong đợi giá trị có thể bị cập nhật.

## 5.4. Return và Exit

Khai báo kiểu trả về sau dấu **:** của danh sách tham số. **exit(value)** trả về giá trị; **exit;** dùng để thoát sớm trong procedure không cần trả giá trị. Không nên lẫn lộn kết quả Boolean kiểu “thành công/thất bại” với lỗi nghiệp vụ có thể cần Error.

```
procedure IsValidWeight(Weight: Decimal): Boolean
begin
    exit((Weight &gt; 0) and (Weight &lt;= 500));
end;
```

## 5.5. Tách logic tốt hơn: ví dụ

Thay vì viết một procedure dài 200 dòng vừa lấy dữ liệu vừa duyệt vừa ghi, hãy tổ chức thành: **ValidateShipment** (không ghi), **CalculateFreight** (tính), **ApproveShipment** (điều phối và ghi), **OnAfterShipmentApproved** (event publisher). Tên thể hiện mục tiêu sẽ giúp đồng nghiệp hiểu mã nhanh.

> **Khi làm trong doanh nghiệp**
> Procedure liên quan kế toán, posting, tồn kho nên sử dụng API/hàm chuẩn của Business Central thay vì cập nhật trực tiếp các bảng ledger. Tách rõ “đọc/tính” và “ghi dữ liệu” để giảm rủi ro giao dịch.

### Tự kiểm tra

- Trong procedure CalculateFreight, biến Weight có bị thay đổi không?
- Từ khóa var trước một parameter tạo khác biệt gì?
- Helper chỉ gọi trong codeunit nên để local hay public?

**Gợi ý đáp án:** Weight không bị đổi bởi ví dụ; var cho phép thay đổi biến gốc; helper thuần nội bộ nên local.

# 6. CODEUNIT: LỚP LOGIC CỦA EXTENSION

## 6.1. Codeunit là gì?

Codeunit là một object dùng để đóng gói nghiệp vụ, chứa procedure, trigger OnRun và có thể có biến toàn cục. Nó tương tự một service nhỏ, không nhất thiết có giao diện hay bảng riêng. Khuyến nghị đặt các quy tắc dùng chung, tính phí, kiểm tra và điều phối trong codeunit.

```
codeunit 50120 "Freight Service"
{
    procedure GetFee(Weight: Decimal;
                     UnitRate: Decimal): Decimal
    begin
        if Weight &lt;= 0 then
            Error('Weight must be positive');
        exit(Round(Weight * UnitRate, 0.01));
    end;
}
```

## 6.2. Gọi procedure từ codeunit khác

```
codeunit 50121 "Freight Demo Runner"
{
    trigger OnRun()
    var
        FreightService: Codeunit "Freight Service";
        Fee: Decimal;
    begin
        Fee := FreightService.GetFee(2.5, 20000);
        if GuiAllowed then
            Message('Phi giao hang: %1', Fee);
    end;
}
```

Ở đây FreightService là biến kiểu codeunit; bạn gọi các procedure public phù hợp. **Codeunit.Run** và thuộc tính **RunObject** có thể chạy logic OnRun, nhưng không thay thế lời gọi trực tiếp tới một procedure có tên.

## 6.3. OnRun khác với procedure tự định nghĩa

| OnRun | Procedure public |
|---|---|
| Điểm vào khi codeunit được Run | Được gọi tường minh bằng tên |
| Thuận cho tác vụ một bước hoặc batch | Thuận cho API logic tái sử dụng |
| Không nên gói quá nhiều side effect khó đoán | Có thể tách theo các hoạt động nhỏ |

## 6.4. Codeunit và phân quyền

Quyền chạy codeunit và quyền thao tác bảng không luôn giống nhau. Với nghiệp vụ cần quyền đặc biệt, cấu hình Permissions của object, permission sets và kiểm tra vai trò người dùng; đừng tăng quyền bằng giải pháp tùy tiện. Một codeunit có quyền truy cập bảng nhạy cảm phải có cơ chế kiểm soát nghiêm ngặt.

## 6.5. Những nguyên tắc tổ chức tốt

- Đặt tên theo chức năng: FreightCalculator, ShipmentValidator, CustomerSyncService.

- Không cho mọi logic vào một “CommonUtils” khổng lồ.
- Không phụ thuộc GUI ở codeunit có khả năng gọi từ API/job queue.
- Phân biệt phương thức chỉ đọc với phương thức ghi dữ liệu.
- Mỗi lỗi phải có ngữ cảnh; các thao tác có tác động dữ liệu cần kiểm thử và logging.

### Tự kiểm tra

- Để gọi GetFee của Freight Service từ một codeunit khác, cần khai báo biến kiểu gì?
- RunObject chạy procedure GetFee hay OnRun?
- Tại sao tách codeunit kiểm tra khỏi codeunit ghi dữ liệu?

**Gợi ý đáp án:** Khai báo Codeunit "Freight Service"; RunObject gọi OnRun; tách lớp giúp dễ kiểm thử và tránh trạng thái ghi nửa chừng.

# 7. TRIGGERS, EVENTS VÀ KIẾN TRÚC MỞ RỘNG

## 7.1. Trigger: tự chạy ở thời điểm nhất định

Trigger là đoạn AL được nền tảng gọi khi điều kiện hệ thống xảy ra: thêm/sửa/xóa dữ liệu, kiểm tra trường nhập, mở page hoặc nhấn action. Trigger khác procedure: procedure phải được gọi (trừ khi tích hợp vào trigger), còn trigger gắn với vòng đời đối tượng.

| Nhóm | Trigger hay gặp | Khi sử dụng |
|---|---|---|
| Table | OnInsert, OnModify, OnDelete, OnRename | Kiểm tra trước khi thay đổi bản ghi |
| Field trong table | OnValidate, OnLookup | Ràng buộc và tìm kiếm trường |
| Page | OnInit, OnOpenPage, OnAfterGetRecord | Khởi tạo, hiển thị thông tin |
| Action trên page | OnAction | Người dùng nhấn nút |
| Codeunit | OnRun | Codeunit được gọi chạy |

## 7.2. OnValidate và Validate

Khi người dùng cập nhật một trường qua giao diện, logic OnValidate thích hợp để kiểm tra giá trị. Nếu trong AL bạn chỉ gán **Record.Field := Value**, bạn không mặc định gọi OnValidate. Muốn kiểm tra và thực thi logic OnValidate của trường, dùng **Record.Validate(Field, Value)**. Đây là khác biệt cực kỳ quan trọng trong ERP.

```
// KHONG chay OnValidate cua truong
Shipment."Weight (Kg)" := 2.5;

// Co chay OnValidate cua truong
Shipment.Validate("Weight (Kg)", 2.5);
```

## 7.3. Tableextension và pageextension

Để mở rộng bảng chuẩn, tạo tableextension thay vì sửa trực tiếp table nền. Ví dụ thêm thông tin phân loại khách hàng. Trong dự án thật, cần khảo sát các trường đã có sẵn, quyền và phân loại dữ liệu.

```
tableextension 50130 "Customer Delivery Ext" extends Customer
{
    fields
    {
        field(50130; "Preferred Delivery Window"; Text[30])
        {
            Caption = 'Preferred delivery window';
            DataClassification = CustomerContent;
        }
    }
}
```

Sau đó có thể tạo pageextension để đưa trường mới lên Customer Card. Lưu ý layout addlast/addafter phải trỏ vào một control/area có thật theo symbols của phiên bản ứng dụng.

## 7.4. Kiến trúc event: publisher và subscriber

Event giúp hai phần mã giao tiếp lỏng: **publisher** công bố điểm mở rộng; **subscriber** đăng ký và được gọi khi event phát ra. Trong Business Central, phần lớn công việc tùy biến là **subscribe các event chuẩn**, tránh thay đổi trực tiếp base application.

Phân biệt: **OnBefore...** thường phục vụ kiểm tra hoặc điều chỉnh đầu vào; **OnAfter...** thường dành cho tác vụ sau khi một bước hoàn tất. Tuy nhiên, tên event không bảo đảm transaction đã commit: phải xem vị trí phát event trong mã nguồn.

## 7.5. Tự định nghĩa một event

```
codeunit 50131 "Shipment Event Demo"
{
    procedure ApproveShipment(ShipmentNo: Code[20])
    begin
        if ShipmentNo = '' then
            Error('Shipment number is required');
        // Business logic would go here.
        OnAfterShipmentApproved(ShipmentNo);
    end;
```

```
    [IntegrationEvent(false, false)]
    local procedure OnAfterShipmentApproved(
        ShipmentNo: Code[20])
    begin
    end;
}
```

## 7.6. Đăng ký subscriber

```
codeunit 50132 "Shipment Event Handler"
{
    [EventSubscriber(ObjectType::Codeunit,
       Codeunit::"Shipment Event Demo",
       'OnAfterShipmentApproved', '', false, false)]
    local procedure AfterApproval(ShipmentNo: Code[20])
    begin
        // Demo only. Replace by a real audit mechanism.
        if GuiAllowed then
            Message('Da duyet: %1', ShipmentNo);
    end;
}
```

EventSubscriber khai báo loại object, object nguồn, tên event và các tùy chọn quyền/license. Các parameter của subscriber phải khớp publisher. Đừng dựa vào thứ tự chạy giữa nhiều subscriber; code phải chạy đúng dù các subscriber khác có mặt.

## 7.7. Event cho ứng dụng chuẩn

Khi cần mở rộng một hành vi chuẩn, dùng **Event Recorder** hoặc danh sách event trong VS Code để tìm event phù hợp; kiểm tra signature và đối tượng phát. Trong AL VS Code, tổ hợp **Shift+Alt+E** được Microsoft giới thiệu để tìm/chèn event subscriber tại vị trí con trỏ.

> **Thực hành an toàn**
> Mỗi subscriber phải ngắn gọn, có mục tiêu và tránh lặp vô hạn: sự kiện sửa trường  subscriber sửa cùng trường  phát lại sự kiện. Không tự viết lại posting hoặc thay đổi ledger nếu có API/event chính thức.

### Tự kiểm tra

- Đâu là khác biệt giữa OnAction và procedure thông thường?
- Vì sao câu gán trực tiếp không tương đương Record.Validate?
- Subscriber có nên giả định luôn chạy đầu tiên không?

**Gợi ý đáp án:** OnAction tự chạy khi người dùng nhấn action; Validate thực thi OnValidate; không được giả định thứ tự subscriber.

# 8. INTERFACE: THAY THẾ CÁCH TRIỂN KHAI

## 8.1. Interface giải quyết vấn đề gì?

Giả sử công ty có nhiều cách tính cước theo đối tác: nội thành, liên tỉnh, nhanh. Nếu dùng một khối if/case rất lớn, mỗi chính sách mới buộc sửa mã chính. Interface định nghĩa một **hợp đồng phương thức**; các codeunit khác nhau cùng thực hiện hợp đồng đó, từ đó giảm sự phụ thuộc vào một cách tính cụ thể.

## 8.2. Khai báo interface và triển khai

```
interface "IFreightCalculator"
{
    procedure Calculate(Weight: Decimal): Decimal;
}
```

```
codeunit 50140 "Local Freight" implements "IFreightCalculator"
{
    procedure Calculate(Weight: Decimal): Decimal
    begin
        exit(15000 + Weight * 2000);
    end;
}
```

```
codeunit 50141 "Express Freight" implements "IFreightCalculator"
{
    procedure Calculate(Weight: Decimal): Decimal
    begin
        exit(30000 + Weight * 5000);
    end;
}
```

Ở đây cả hai codeunit đều có Calculate(Decimal): Decimal. Compiler kiểm tra chữ ký của procedure đáp ứng interface. Interface không chứa thân xử lý mà xác định những khả năng bắt buộc của implementation.

## 8.3. Dùng cùng một kiểu để gọi các thuật toán khác nhau

```
procedure QuoteFreight(IsExpress: Boolean;
                       Weight: Decimal): Decimal
var
    Calculator: Interface "IFreightCalculator";
    LocalCalc: Codeunit "Local Freight";
    ExpressCalc: Codeunit "Express Freight";
begin
    if IsExpress then
        Calculator := ExpressCalc
    else
        Calculator := LocalCalc;
    exit(Calculator.Calculate(Weight));
end;
```

Phần gọi không cần biết công thức bên trong LocalCalc hay ExpressCalc. Khi thêm dịch vụ mới, bạn triển khai codeunit mới theo interface và thay đổi cách lựa chọn implementation, thay vì sửa mọi nơi gọi phép tính.

## 8.4. Interface và enum có thể kết hợp

Trong kiến trúc mở rộng hơn, một enum có thể ánh xạ từng giá trị sang implementation của interface thông qua thuộc tính Implementation. Điều này hữu ích khi thêm phương thức xử lý theo loại đối tác hoặc loại giao dịch. Lộ trình Microsoft Learn còn đề cập type testing/casting và mở rộng interface. Người mới nên nắm chắc ví dụ hai codeunit trước, rồi mới học khả năng nâng cao này.

## 8.5. Khi nào chưa cần interface?

- Nếu logic chỉ một công thức đơn giản, một codeunit/procedure thường dễ đọc hơn.
- Dùng interface khi thực sự có nhiều chiến lược có thể thay thế hoặc các extension cần bổ sung phương thức mà không sửa lõi.
- Không tạo interface cho mọi function, tránh làm dự án nhỏ trở nên phức tạp không cần thiết.
- Chữ ký interface là hợp đồng: đổi tham số hoặc kiểu trả về có thể ảnh hưởng mọi implementation.

### Tự kiểm tra

- Hai đơn vị vận chuyển dùng cùng phương thức Calculate nhưng công thức khác nhau: interface có phù hợp không?
- Nếu chỉ có một hàm cộng phí đơn giản, có cần interface không?

**Gợi ý đáp án:** Phù hợp khi dự kiến có nhiều chiến lược thay thế; không nhất thiết cần interface với logic một lần đơn giản.

# 9. THAO TÁC DỮ LIỆU: RECORD VÀ CƠ SỞ DỮ LIỆU

## 9.1. Dùng Record để làm việc với bảng

Biến **Record "Table Name"** đại diện cho các dòng trong một bảng, cùng bộ lọc, thứ tự và các method truy cập. Viết AL nghĩa là tuân theo quy tắc dữ liệu của hệ thống, không trực tiếp phát lệnh SQL từ extension SaaS.

## 9.2. Get, FindFirst, FindLast, FindSet và IsEmpty

| Method | Ý nghĩa | Trường hợp điển hình |
|---|---|---|
| Get(PK) | Lấy bản ghi theo khóa chính | Tìm một khách hàng theo mã |
| FindFirst / FindLast | Lấy dòng đầu/cuối của tập lọc | Tìm hóa đơn đầu tiên trong kỳ |
| FindSet | Duyệt toàn bộ tập bản ghi đã lọc | Tính tổng theo lô |
| Next | Chuyển sang dòng tiếp theo | Trong repeat ... until |
| IsEmpty | Kiểm tra không có dữ liệu | Kiểm tra điều kiện tồn tại |

```
procedure FindCustomer(CustomerNo: Code[20]): Text
var
    Customer: Record Customer;
begin
    if not Customer.Get(CustomerNo) then
        Error('Khong tim thay khach hang %1', CustomerNo);
    exit(Customer.Name);
end;
```

Nếu Get không có nhánh kiểm tra Boolean và bản ghi không tồn tại, có thể phát sinh runtime error. Với FindSet, hãy đặt lọc phù hợp **trước** khi lấy dữ liệu và kiểm tra nếu tập rỗng.

## 9.3. SetRange, SetFilter, SetCurrentKey

**SetRange** phù hợp kiểm tra bằng hoặc trong khoảng; **SetFilter** cho điều kiện linh hoạt (>, <, ký tự đại diện, OR...); **SetCurrentKey** gợi ý cách sắp xếp/truy cập theo key phù hợp. Việc gọi SetCurrentKey không đảm bảo ép SQL dùng một index vật lý cụ thể.

```
Customer.SetRange("Country/Region Code", 'VN');
Customer.SetFilter(Name, '@H*');
Customer.SetCurrentKey(Name);
if Customer.FindSet() then
    repeat
        // Xu ly khach hang thoa loc
    until Customer.Next() = 0;
```

## 9.4. Thêm dữ liệu với Init, Validate, Insert

Để tạo bản ghi mới, thường **Init** rồi thiết lập trường, gọi Validate trên các trường có nghiệp vụ, và **Insert(true)** nếu cần table OnInsert trigger. Mặc định tham số RunTrigger của Insert/Modify/Delete là false: phải hiểu mình có muốn chạy table trigger không.

```
NewRequest.Init();
NewRequest."No." := 'REQ-0001';
NewRequest.Validate("Customer No.", '10000');
NewRequest."Description" := 'Test request';
NewRequest.Insert(true);
```

Ví dụ giả định NewRequest là biến Record của một bảng tùy chỉnh có các trường nêu trên. Không đưa trực tiếp vào dữ liệu thật; mã số có thể cần được cấp bằng number series của doanh nghiệp.

## 9.5. Modify, ModifyAll, Delete, DeleteAll

Sau khi lấy bản ghi, cập nhật trường rồi gọi Modify(true) nếu cần OnModify. ModifyAll thay đổi hàng loạt có lọc; DeleteAll xóa hàng loạt có lọc. Đây là những lệnh có rủi ro cao: luôn xác nhận filter, quyền, số dòng và điều kiện giao dịch. Không viết DeleteAll trống bộ lọc trên bảng nghiệp vụ trong môi trường production.

```
if Request.Get('REQ-0001') then begin
    Request.Validate(Status, Request.Status::Approved);
    Request.Modify(true);
end;
```

Cú pháp này minh họa với một bảng tùy chỉnh có trường enum Status. Trước khi Modify hoặc Delete, cần xác minh bản ghi đúng loại/đúng công ty, tránh cập nhật sai dữ liệu nhiều tenant/company.

## 9.6. TestField, FieldError, Validate

Ba phương thức thường xuất hiện trong code nghiệp vụ: **TestField** yêu cầu trường có giá trị hoặc đúng giá trị; **FieldError** nêu lỗi cho trường; **Validate** gán giá trị và chạy trigger OnValidate. Trong Business Central, dùng phép gán trực tiếp cho tất cả trường có thể bỏ qua một số quy tắc quan trọng.

## 9.7. FlowField, CalcFields, CalcSums

FlowField là trường tính toán, thường không lưu kết quả như trường thông thường. Khi đọc FlowField trong AL, gọi **CalcFields** hoặc thiết lập **SetAutoCalcFields** trước khi dùng. **CalcSums** có thể tính tổng theo các trường và điều kiện lọc tương thích; không thay thế việc xác minh tính đúng của số liệu nghiệp vụ.

```
if Customer.Get('10000') then begin
    Customer.CalcFields(Balance);
    Message('So du khach hang: %1', Customer.Balance);
end;
```

## 9.8. Khóa dữ liệu và hiệu năng

Nhiều người dùng ERP có thể đồng thời sửa một đơn hàng. Nền tảng chọn mức cô lập và hành vi khóa, nhưng AL cũng có công cụ như **LockTable** và **ReadIsolation** cho các tình huống cần thiết. Chỉ thay đổi khóa/cô lập khi đã hiểu transaction, tính nhất quán và có benchmark thực tế; khóa quá rộng khiến người dùng khác phải chờ.

- Lọc ở cấp database trước khi FindSet; tránh tải tất cả rồi tự lọc bằng vòng lặp.
- Không dùng FindFirst liên tiếp trong vòng lặp khi có thể tổ chức truy vấn hiệu quả hơn.
- Thao tác sửa/xóa hàng loạt: luôn có điều kiện lọc, log, backup/rollback và thử nghiệm.
- Chú ý permission sets, company context và transaction boundaries; không tự động COMMIT tùy tiện.
- Không sửa trực tiếp ledger/accounting entries thay cho nghiệp vụ posting chuẩn của Business Central.

### Tự kiểm tra

- Muốn tìm khách hàng theo khóa chính, nên Get hay FindSet?
- Vì sao gán trường bằng := khác Validate(Field, Value)?

- Để đọc FlowField Balance, phương thức nào quan trọng?
- DeleteAll không filter có an toàn không?

**Gợi ý đáp án:** Get; Validate chạy logic OnValidate; CalcFields; không an toàn vì có thể xóa mọi bản ghi của table theo ngữ cảnh.

# 10. PHÁT TRIỂN ĐA NGÔN NGỮ

## 10.1. Vì sao extension cần translation?

Business Central phục vụ các tổ chức nhiều ngôn ngữ. Một extension có label, caption, tooltip và thông báo lỗi; nếu viết hard-code tất cả bằng một ngôn ngữ, người dùng bản địa có thể khó thao tác, đồng thời việc bảo trì gặp trở ngại.

## 10.2. Caption và Label

Dùng thuộc tính **Caption** để đặt nhãn trên table/page/field/action. Dùng biến **Label** cho thông điệp và chuỗi dùng trong xử lý. AL project có thể trích xuất chuỗi dịch sang XLF; dịch ở XLF để giữ mã nguồn ổn định.

```
var
    InvalidWeightErr: Label 'Weight must be positive';

procedure CheckWeight(Weight: Decimal)
begin
    if Weight &lt;= 0 then
        Error(InvalidWeightErr);
end;
```

## 10.3. Cấu hình TranslationFile và sinh tệp XLF

Trong **app.json**, khai báo feature **TranslationFile** (tùy cấu hình/version). Khi build/package extension, hệ thống tạo tệp ***.g.xlf** trong thư mục Translations. Sao chép/đặt tên tệp theo locale phù hợp, chẳng hạn **vi-VN.xlf**, rồi điền các cặp source/target và thông tin language. Tránh sửa thủ công tệp .g.xlf do công cụ sinh ra.

```
{
  "features": ["TranslationFile"]
}
```

Đoạn JSON chỉ minh họa thuộc tính cần có; không phải toàn bộ app.json. Hãy giữ nguyên các metadata như id, publisher, name, version, application, platform và các trường project đang có.

## 10.4. Cấu trúc tối giản của bản dịch XLF

```
&lt;trans-unit id="sample-weight-message"&gt;
  &lt;source&gt;Weight must be positive&lt;/source&gt;
  &lt;target state="translated"&gt;Can nang phai lon hon 0&lt;/target&gt;
&lt;/trans-unit&gt;
```

Đây chỉ là **một phần tử minh họa**, không phải một file XLF hoàn chỉnh. Tệp thật còn có namespace, version, file và ID do công cụ sinh; phải giữ nguyên định danh và cấu trúc để mapping chính xác. Cần kiểm tra target-language theo mã locale mà bạn dùng.

## 10.5. Dịch tự động và kiểm tra chất lượng

Microsoft Learn có phần dùng Microsoft Multilingual App Toolkit và Azure Translator Text để hỗ trợ dịch. Dịch tự động giúp tạo bản nháp nhưng không thay thế kiểm tra thuật ngữ ERP: posting, vendor, item ledger, transfer order, purchase receipt thường có ngữ nghĩa nghiệp vụ rất cụ thể.

- Tạo danh mục thuật ngữ thống nhất; không dịch cùng khái niệm theo nhiều cách.
- Kiểm tra placeholder %1, %2 và đặc trưng viết hoa trong Label.
- Kiểm thử giao diện có chuỗi dài, trường bị cắt, ký tự Unicode và nội dung tooltip.

- Xác nhận dịch vụ Azure hoặc công cụ ngoài đáp ứng chính sách dữ liệu và chi phí của công ty.
- Kiểm tra translation sau mỗi đợt đổi label hoặc cập nhật nền tảng.

### Tự kiểm tra

- Nên sửa trực tiếp tệp *.g.xlf đã được sinh tự động không?
- Label và Caption thường dùng để làm gì?
- Vì sao phải kiểm tra placeholder khi dịch?

**Gợi ý đáp án:** Không; Caption cho nhãn giao diện, Label cho thông điệp/chuỗi; placeholder bảo đảm thông tin động vẫn hiển thị đúng.

# 11. DỰ ÁN CUỐI KHÓA: DELIVERY REVIEW

## 11.1. Bài toán nghiệp vụ

Xây một extension nhỏ để quản lý yêu cầu kiểm tra giao hàng. Mỗi yêu cầu có mã, khách hàng, cân nặng, trạng thái và thời điểm phê duyệt. Nhân viên nhập dữ liệu trên page. Nút **Approve** gọi codeunit kiểm tra trường bắt buộc, cập nhật trạng thái và phát event. Dự án gom các kiến thức: table, enum, page, codeunit, procedure, Validate, Modify và event subscriber.

> **Phạm vi của bài lab**
> Đây là bài thực hành trong sandbox **không phải** giải pháp logistics đầy đủ: chưa có number series tự động, quyền ứng dụng, luồng phê duyệt nhiều cấp, lịch sử thay đổi hoặc tích hợp API. Không publish sản phẩm này lên production khi chưa hoàn thiện các phần đó.

## 11.2. Cấu trúc thư mục đề xuất

```
DeliveryReview/
  app.json
  launch.json
  src/
    DeliveryReviewStatus.Enum.al
    DeliveryReview.Table.al
    DeliveryReviewList.Page.al
    DeliveryReviewService.Codeunit.al
    DeliveryReviewSubscriber.Codeunit.al
  Translations/
```

## 11.3. File 1 - Enum trạng thái

```
enum 50171 "Delivery Review Status"
{
    Extensible = false;
    value(0; Open) { Caption = 'Open'; }
    value(1; Approved) { Caption = 'Approved'; }
}
```

## 11.4. File 2 - Table lưu yêu cầu

```
table 50170 "Delivery Review"
{
    Caption = 'Delivery Review';
    DataClassification = CustomerContent;

    fields
    {
        field(1; "No."; Code[20])
        {
            Caption = 'No.';
        }
        field(2; "Customer No."; Code[20])
        {
            Caption = 'Customer No.';
            TableRelation = Customer."No.";
        }
        field(3; "Weight (Kg)"; Decimal)
        {
            Caption = 'Weight (Kg)';
            trigger OnValidate()
            begin
                if "Weight (Kg)" &lt; 0 then
                    Error('Weight cannot be negative');
            end;
        }
        field(4; Status; Enum "Delivery Review Status")
        {
            Caption = 'Status';
        }
        field(5; "Last Checked At"; DateTime)
        {
            Caption = 'Last Checked At';
            Editable = false;
        }
    }

    keys
    {
        key(PK; "No.") { Clustered = true; }
    }
}
```

Khóa chính là No.; trường Customer No. có liên kết bảng Customer. Ở trigger OnValidate, ví dụ chỉ cấm cân nặng âm để người dùng có thể lưu nháp cân nặng 0; bước Approve sẽ yêu cầu cân nặng dương.

## 11.5. File 3 - Codeunit kiểm tra và duyệt

```
codeunit 50172 "Delivery Review Service"
{
    procedure Approve(var Review: Record "Delivery Review")
    begin
        Review.TestField("Customer No.");
        Review.TestField("Weight (Kg)");

        if Review."Weight (Kg)" &lt;= 0 then
            Review.FieldError("Weight (Kg)",
                'must be greater than zero');
```

```
        if Review.Status = Review.Status::Approved then
            Error('Request is already approved');

        Review.Validate(Status, Review.Status::Approved);
        Review."Last Checked At" := CurrentDateTime();
        Review.Modify(true);

        OnAfterApproved(Review."No.");
    end;
```

```
    [IntegrationEvent(false, false)]
    local procedure OnAfterApproved(ReviewNo: Code[20])
    begin
    end;
}
```

Mục đích của **var Review** là cho phép method sửa bản ghi truyền vào, đồng thời lưu lên bảng bằng Modify(true). Trong hệ thống thực tế, nên kiểm tra quyền và chặn phê duyệt lại theo cả trạng thái/sửa đồng thời. Event gọi sau Modify trong cùng quá trình, **không mặc định có nghĩa commit xong**.

## 11.6. File 4 - Page nhập và duyệt

```
page 50173 "Delivery Review List"
{
    PageType = List;
    SourceTable = "Delivery Review";
    ApplicationArea = All;
    UsageCategory = Lists;
    Caption = 'Delivery Reviews';
    Editable = true;
```

```
    layout
    {
        area(Content)
        {
            repeater(General)
            {
                field("No."; Rec."No.")
                {
                    ApplicationArea = All;
                }
                field("Customer No."; Rec."Customer No.")
                {
                    ApplicationArea = All;
                }
                field("Weight (Kg)"; Rec."Weight (Kg)")
                {
                    ApplicationArea = All;
                }
                field(Status; Rec.Status)
                {
                    ApplicationArea = All;
                    Editable = false;
                }
                field("Last Checked At"; Rec."Last Checked At")
                {
                    ApplicationArea = All;
                }
            }
        }
    }
```

```
    actions
    {
        area(Processing)
        {
            action(Approve)
            {
                ApplicationArea = All;
                Caption = 'Approve';
                Image = Approve;
```

```
                trigger OnAction()
                var
                    Service: Codeunit "Delivery Review Service";
                begin
                    Service.Approve(Rec);
                    CurrPage.Update(false);
                end;
            }
        }
    }
}
```

Hai block trên là **hai phần liên tiếp của cùng file Page**: phần thứ hai tiếp nối sau dấu đóng của layout, trước dấu đóng của page. Nhớ ghép vào một object duy nhất, không tách thành hai page.

## 11.7. File 5 - Subscriber minh họa

```
codeunit 50174 "Delivery Review Events"
{
    [EventSubscriber(ObjectType::Codeunit,
        Codeunit::"Delivery Review Service",
        'OnAfterApproved', '', false, false)]
    local procedure HandleApproved(ReviewNo: Code[20])
    begin
        // Replace with actual audit/notification logic.
        if GuiAllowed then
            Message('Approved review: %1', ReviewNo);
    end;
}
```

Message chỉ là dấu hiệu để quan sát event trong sandbox. Không dùng event subscriber này để bảo đảm hệ thống có audit log. Trong sản phẩm thực tế, log cần được thiết kế, phân quyền và kiểm thử độc lập.

## 11.8. Quy trình chạy thử từng bước

- Tạo sandbox/project, đặt ID trong dải được phép, tải symbols và bảo đảm base application có Customer table.
- Tạo lần lượt 5 file .al. Gõ đúng tên các object và tách phần Page theo hướng dẫn trên.
- Build/package extension; sửa lỗi compiler, sau đó publish sang sandbox.
- Trong Business Central tìm **Delivery Reviews**; thêm mã DR-001, chọn một Customer có thật và cân nặng 2.5.
- Nhấn Approve; kiểm tra Status=Approved và Last Checked At được ghi.
- Kiểm thử lỗi: không có Customer No., Weight = 0, nhấn Approve hai lần; ghi lại thông báo hiển thị.
- Thêm kiểm thử: sửa thử Weight thành giá trị âm; xác nhận OnValidate từ table ngăn nhập.
- Thử thay subscriber bằng ghi log vào một bảng mới (bài mở rộng), lưu ý transaction/permission.

## 11.9. Tiêu chí tự chấm điểm

| Tiêu chí | Đạt nếu |
|---|---|
| Build | Không còn lỗi compile/symbol/object ID |
| Data | Tạo và lưu được bản ghi nháp |
| Validation | Không cho duyệt nếu thiếu dữ liệu |
| State | Approve chuyển trạng thái và lưu thời gian |
| Event | Subscriber được gọi đúng và không phá quy trình |
| Quality | Codeunit tách khỏi page, dễ đọc, tên nhất quán |
| Safety | Đã thử trên sandbox, không đụng ledger |

# 12. TỰ KIỂM TRA, LỖI THƯỜNG GẶP & LỘ TRÌNH HỌC

## 12.1. Mười lỗi người mới thường gặp

| Hiện tượng | Nguyên nhân / cách xử lý |
|---|---|
| AL: Download Symbols thất bại | Kiểm tra tenant/environment, login, quyền, version |
| Object not found | Symbol chưa tải/không khớp app dependencies |
| ID duplicated | Trùng ID, phải dùng dải được cấp |
| Field does not exist | Sai tên, ngữ cảnh Rec, symbols khác phiên bản |
| Procedure inaccessible | Procedure local/internal hoặc object chưa hợp lệ |
| Không chạy OnValidate | Đã gán := thay vì Record.Validate |
| Insert/Modify bỏ qua trigger | RunTrigger mặc định false, cần cân nhắc true |
| Không thấy FlowField | Chưa gọi CalcFields/SetAutoCalcFields |
| Subscriber không chạy | Sai object/event/signature, chưa phát event |
| API thất bại vì Dialog | Code giả định có GUI khi đang chạy nền |

## 12.2. Câu hỏi ôn tập tổng hợp

**01.** Biến local khác global như thế nào?

**02.** Code[n] khác Text[n] ra sao?

**03.** Vì sao enum thường dễ mở rộng hơn option?

**04.** Array và List bắt đầu index bằng mấy?

**05.** Khi nào nên dùng Dictionary?

**06.** Để gán trong AL dùng toán tử nào?

**07.** Phép so sánh bằng sử dụng ký hiệu gì?

**08.** Vì sao có begin ... end sau if?

**09.** Repeat ... until dừng khi nào?

**10.** WorkDate khác Today ở điểm nào?

**11.** Evaluate trả ý nghĩa gì khi được kiểm tra Boolean?

**12.** Trước khi Confirm trong API cần lưu ý gì?

**13.** Error khác Message ở điểm nào?

**14.** TryFunction có tự rollback database write không?

**15.** Tại sao nên tách business logic sang codeunit?

**16.** var trên tham số procedure có tác dụng gì?

**17.** OnRun có vai trò gì?

**18.** OnValidate liên quan đến Record.Validate ra sao?

**19.** Publisher và subscriber khác nhau thế nào?

**20.** Vì sao không nên giả định thứ tự chạy subscriber?

**21.** Interface có chứa logic xử lý bên trong không?

**22.** Get khác FindSet ở trường hợp sử dụng nào?

**23.** Lọc dữ liệu trước khi duyệt có lợi gì?

**24.** Làm sao tính FlowField trong code?

**25.** Có nên sửa trực tiếp .g.xlf đã được sinh không?

## 12.3. Đáp án ngắn

01. Local trong procedure; global trong object.
02. Code chuẩn hóa mã (thường uppercase/trim); Text lưu chuỗi thông thường.
03. Enum là object tái sử dụng và có thể được enumextension mở rộng khi cho phép.
04. Bắt đầu bằng 1.
05. Khi cần tra cứu value theo key duy nhất.
06. := .
07. = .
08. Để nhóm nhiều câu lệnh vào một nhánh.
09. Khi biểu thức sau until thành true.
10. WorkDate là ngày làm việc ứng dụng/người dùng; Today là ngày hiện tại.
11. True nếu chuyển thành công, false nếu thất bại.
12. GUI có thể không khả dụng; kiểm tra GuiAllowed hoặc dùng cơ chế API.
13. Error dừng xử lý; Message thông báo.
14. Không.
15. Để tái sử dụng, kiểm thử, duy trì và quản lý quyền.
16. Truyền tham chiếu và có thể thay đổi biến ở phía gọi.
17. Chạy khi codeunit được gọi chạy.
18. Validate(Field, Value) kích hoạt logic OnValidate của trường.
19. Publisher phát sự kiện, subscriber nhận và xử lý.
20. Mã phải độc lập với subscriber khác; thứ tự không được bảo đảm.
21. Không; chỉ khai báo hợp đồng phương thức.
22. Get theo khóa chính; FindSet lấy một tập bản ghi theo filter.
23. Giảm dữ liệu truy xuất và thường cải thiện hiệu năng.
24. CalcFields hoặc SetAutoCalcFields.
25. Không; tạo/duy trì XLF bản dịch riêng.

## 12.4. Kế hoạch 14 ngày để học chắc

| Ngày | Trọng tâm & bài thực hành |
|---|---|
| 1–2 | Cài môi trường, AL: Go!, symbols, codeunit Hello Demo |
| 3 | Biến, kiểu, enum và List/Dictionary |
| 4 | If/case, vòng lặp; kiểm tra biểu thức |
| 5 | Hàm chuỗi, ngày, số; thử Evaluate và Format |
| 6 | Error, TestField, FieldError, TryFunction |
| 7 | Procedure, local/global, var, return |
| 8 | Codeunit service, tách logic tính phí |
| 9 | Table và page triggers; phân biệt Validate với := |
| 10 | Event publisher/subscriber, Event Recorder |
| 11 | Interface và 2 chiến lược tính phí |
| 12 | Record CRUD, filter, FlowField, hiệu năng |
| 13 | Label và XLF; chạy dự án Delivery Review |
| 14 | Viết test case âm, review, giải 25 câu ôn tập |

## 12.5. Thuật ngữ tiếng Anh - tiếng Việt

| Thuật ngữ | Cách hiểu nhanh |
|---|---|
| Extension | Gói bổ sung/mở rộng chức năng Business Central |
| Object | Đối tượng AL (table, page, codeunit...) |
| Record | Bản ghi dữ liệu của một bảng |
| Trigger | Đoạn mã tự chạy do một sự kiện hệ thống |
| Event publisher | Nơi công bố sự kiện |
| Event subscriber | Đoạn mã đăng ký nhận sự kiện |
| Procedure | Thủ tục/hàm |
| FlowField | Trường có giá trị tính toán |
| Validate | Thiết lập/kiểm tra trường kèm logic OnValidate |
| Sandbox | Môi trường kiểm thử không phải production |
| XLF / XLIFF | Định dạng tệp phục vụ dịch đa ngôn ngữ |

# NGUỒN HỌC VÀ CÁCH ĐỌC TIẾP

Giáo trình này được biên soạn độc lập theo cấu trúc **10 mô-đun chính thức** của Microsoft Learn, bổ sung diễn giải, ví dụ, bài tập và dự án mẫu tự tạo. Tài liệu hướng đến học tập; không phải hướng dẫn nâng cấp chính thức cho một phiên bản Business Central cụ thể. Kiểm tra Microsoft Learn và symbols của môi trường khi đưa mã vào công việc thực tế.

**Lộ trình 10 học phần** — https://learn.microsoft.com/vi-vn/training/paths/application-foundation-al-language/

**1. Căn bản AL** — https://learn.microsoft.com/vi-vn/training/modules/intro-basics-al-programming/

**2. Câu lệnh AL** — https://learn.microsoft.com/vi-vn/training/modules/al-statements/

**3. Built-in functions** — https://learn.microsoft.com/vi-vn/training/modules/al-built-in-functions/

**4. Xử lý lỗi** — https://learn.microsoft.com/vi-vn/training/modules/handle-errors/

**5. Procedures** — https://learn.microsoft.com/vi-vn/training/modules/create-custom-functions/

**6. Codeunits** — https://learn.microsoft.com/vi-vn/training/modules/work-with-codeunits/

**7. Events và triggers** — https://learn.microsoft.com/vi-vn/training/modules/events-triggers/

**8. Interfaces** — https://learn.microsoft.com/vi-vn/training/modules/business-central-interfaces/

**9. Thao tác dữ liệu** — https://learn.microsoft.com/vi-vn/training/modules/manipulate-data-via-code/

**10. Đa ngôn ngữ** — https://learn.microsoft.com/vi-vn/training/modules/manage-multilanguage-development/

## Sau tài liệu này nên học gì?

- Xây data model: table, FlowField/FlowFilter, keys, relations, data classification.
- Thiết kế giao diện: page, pageextension, FactBox, action, role center.
- Unit tests bằng AL test codeunits, test libraries, CI/CD và extension lifecycle.
- Phân quyền: permission sets, entitlements và bảo vệ dữ liệu.
- Tích hợp Business Central API/REST, OData, webhooks và dịch vụ ngoại vi.
- Posting pipelines, number series, warehouse/sales/purchase workflows và performance profiling.

**Thực hành phù hợp vị trí IT/ERP Support:** thêm trường tùy chỉnh đúng chuẩn, tạo form thu thập lỗi, viết codeunit kiểm tra dữ liệu nhập, log thay đổi, dùng event subscriber thay vì sửa hệ thống chuẩn. Sau đó kết hợp hiểu nghiệp vụ kế toán/kho và kỹ năng đọc log để xử lý yêu cầu người dùng.

> **Kết luận**
> Học AL hiệu quả không dừng ở việc thuộc cú pháp. Mục tiêu là hiểu luồng nghiệp vụ, mô hình dữ liệu, vòng đời object và cách mở rộng an toàn: **đúng dữ liệu, đúng thời điểm, đúng quyền và dễ bảo trì**.